

Blocco 5 – JOIN e cardinalità

Problema, specifica, soluzione e implementazione

Materiali

- Durata: 90 minuti.
- Slide: `slides/blocks/block05_join_cardinalita/block05_join_cardinalita.pdf`
- Ambiente laboratorio: container Docker PostgreSQL `rdsq1-postgres`
- Schema PostgreSQL: `../../sql/01_schema_seed_postgres.sql`
- Script soluzione: `solution.sql`
- Script unico schema + soluzioni: `../../sql/00_schema_and_all_solutions_postgres.sql`

1. Problema in forma informale

Il team vendite vuole leggere insieme clienti, ordini e prodotti, e vuole anche sapere quali ordini non hanno ancora una spedizione.

2. Specifica corretta del problema

- input: tabelle `customers`, `orders`, `order_items`, `products`, `shipments`;
- output: query con granularità dichiarata e join espliciti;
- vincolo: usare `INNER JOIN` quando la relazione è obbligatoria;
- vincolo: usare `LEFT JOIN` solo per collegamenti opzionali come spedizione mancante.

3. Definizione della soluzione

1. costruire prima il join clienti-ordini;
2. aggiungere il dettaglio righe ordine solo quando serve la granularità di dettaglio;
3. usare `LEFT JOIN shipments` per evidenziare ordini senza spedizione;
4. controllare righe base e righe finali con `count(*)`.

4. Implementazione completa o artefatto atteso

La soluzione seguente è caricabile nel container PostgreSQL Docker dopo lo schema di laboratorio.

```
SET search_path TO training;
```

```
SELECT o.order_id, o.order_date, c.full_name, c.country, o.status
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
ORDER BY o.order_date, o.order_id;
```

```
SELECT o.order_id, c.full_name, p.sku, p.product_name,
```

```

        oi.quantity,
        round(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100.0), 2) AS net_amount
FROM order_items oi
JOIN orders o ON o.order_id = oi.order_id
JOIN customers c ON c.customer_id = o.customer_id
JOIN products p ON p.product_id = oi.product_id
ORDER BY o.order_id, p.sku;

SELECT o.order_id, c.full_name, o.order_date, o.status,
       s.shipment_id, s.shipped_at, s.delivered_at
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
LEFT JOIN shipments s ON s.order_id = o.order_id
WHERE s.shipment_id IS NULL
ORDER BY o.order_date, o.order_id;

SELECT o.order_id, count(oi.product_id) AS line_rows
FROM orders o
LEFT JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY o.order_id
ORDER BY line_rows DESC, o.order_id;

```

5. Suggerimenti

- prima di lavorare sulla soluzione, riscrivere la domanda con parole proprie;
- esplicitare input, output, vincoli e assunzioni;
- controllare la granularità del risultato atteso;
- preparare almeno un caso limite da discutere in aula.

6. Criteri di verifica

- la granularità del risultato è dichiarata;
- ogni join segue una chiave o una relazione motivata;
- le righe perse o aggiunte sono spiegate;
- i controlli di conteggio confermano l'aspettativa.

7. Checkpoint

- prima del join bisogna sapere una riga per cosa si vuole ottenere;
- LEFT JOIN non significa “join migliore”: significa relazione opzionale;
- ogni aggregazione dopo join richiede controllo del fan-out.